

2025

2. Define information system. What are its characteristics?

Definition

An **Information System (IS)** is an organized combination of **people, hardware, software, data, procedures, and networks** that collects, processes, stores, and distributes information to support decision-making and control in an organization.

Small points to memorize

- **Input:** collects data
- **Processing:** converts data into useful information
- **Storage:** saves data and information
- **Output:** provides reports/results
- **Feedback:** improves the system
- **Components:** people, hardware, software, data, procedure, network

Characteristics

- **Goal-oriented**
- **Integrated components**
- **Accepts input and gives output**
- **Processes data into information**
- **Provides feedback and control**
- **Supports decision-making**
- **Can be manual or computerized**
- **Requires accuracy, timeliness, and relevance**

1. Goal-Oriented

An information system is designed to achieve specific organizational objectives.

Explanation:

Every IS exists for a purpose—such as managing sales, handling student records, or tracking inventory.

Example:

An e-commerce system like Amazon aims to **sell products efficiently and increase revenue**.

2. Integrated Components

It consists of interconnected parts: hardware, software, data, procedures, and people.

Explanation:

All components must work together; failure in one affects the whole system.

Example:

A web app built with Django (backend), database, and frontend must coordinate to process user requests.

3. Accepts Input and Gives Output

The system takes input, processes it, and produces output.

Explanation:

Input = raw data → Processing → Output = useful information

Example:

User enters login details → system verifies → shows dashboard.

4. Processes Data into Information

Transforms raw data into meaningful, useful information.

Explanation:

Raw data alone has little value until processed.

Example:

Sales data → total revenue report → helps business decisions.

5. Provides Feedback and Control

The system monitors output and adjusts operations accordingly.

Explanation:

Feedback helps improve system performance and correctness.

Example:

If a payment fails, system notifies user and retries or logs error.

6. Supports Decision-Making

Helps managers and users make informed decisions.

Explanation:

Provides reports, analytics, and insights.

Example:

A dashboard showing best-selling products helps decide what to restock.

7. Can Be Manual or Computerized

Information systems can be either manual or automated.

Explanation:

Not all systems require computers, but modern systems are mostly computerized.

Examples:

- **Manual:** Paper-based record system
 - **Computerized:** Database system using PostgreSQL
-

8. Requires Accuracy, Timeliness, and Relevance

Explanation:

- **Accuracy:** Information must be correct
- **Timeliness:** Information must be available when needed
- **Relevance:** Information must be useful for the task

Example:

In a banking system, wrong or delayed transaction data can cause serious issues.

Real-world application

- **Banking system:** takes deposit/withdrawal data, processes it, stores customer records, and shows account balance.
 - **Hospital system:** stores patient records, prescriptions, reports, and appointments.
-

3. What are different types of maintenance and when should they be implemented?

Definition

Software maintenance means modifying and updating software after delivery to keep it useful and efficient.

Types of maintenance

1. Corrective maintenance

- Done to **fix errors/bugs**

- Implemented **when software fails or gives wrong output**

Example: Fixing wrong bill amount in an e-commerce app.

2. Adaptive maintenance

- Done to make software work in a **new environment**
- Implemented **when OS, hardware, laws, or business rules change**

Example: Updating payroll software after new tax rules.

3. Perfective maintenance

- Done to **improve performance or add new features**
- Implemented **when users ask for improvements**

Example: Adding dark mode or faster search to an app.

4. Preventive maintenance

- Done to **avoid future problems**
- Implemented **before failure occurs**

Example: Refactoring old code to reduce future crashes.

Small points to memorize

- **Corrective** = fix
- **Adaptive** = adjust
- **Perfective** = improve
- **Preventive** = prevent

Real-world application

- **Mobile banking app**
 - Corrective: fixing login bug
 - Adaptive: updating for new Android version
 - Perfective: adding UPI history filters
 - Preventive: optimizing database queries
-

4. Define interface. What are different methods of interaction? Explain briefly.

Definition

An **interface** is the point where a **user interacts with a computer system**.

Methods of interaction

1. Command Line Interface (CLI)

- User types commands
- Fast for expert users

Example: Linux terminal

2. Menu-driven interface

- User selects options from menus
- Easy for beginners

Example: ATM machine menu

3. Graphical User Interface (GUI)

- Uses windows, icons, buttons, and menus
- User-friendly and visual

Example: Windows OS, mobile apps

4. Form-based interface

- User enters data into fields/forms
- Common in data entry systems

Example: Registration form, login page

5. Natural language / voice interface

- User speaks or types in normal language
- Easy and interactive

Example: Siri, Google Assistant, ChatGPT

6. Touch interface

- Uses tapping, swiping, pinching
- Common in smartphones and kiosks

Example: Mobile apps, self-service counters

Small points to memorize

- **CLI** = typed commands
- **Menu** = choose options
- **GUI** = icons/buttons
- **Form** = fill fields
- **Voice** = speak
- **Touch** = tap/swipe

Real-world application

- **Railway ticket machine** uses menu-driven and touch interface.
 - **Online banking** uses form-based and GUI interface.
-

5. Prototyping is one of the effective methods to determine the requirement. Do you agree? Justify.

Answer

Yes, I agree. Prototyping is very effective because it helps users and developers understand requirements clearly before final development.

Justification

- Users can **see a sample model**
- Helps in **clarifying vague requirements**
- Reduces misunderstanding
- Encourages user feedback
- Detects missing features early
- Saves cost of later changes
- Improves communication between user and developer

Limitations

- Users may think prototype is final product
- Can increase time if too many changes are made
- May ignore proper documentation

Small points to memorize

- **Shows early model**
- **Gets user feedback**
- **Clarifies requirements**
- **Reduces errors**
- **Saves rework cost**

Real-world application

- Before making a **food delivery app**, developers first create a prototype showing login, menu, cart, and payment screens. Users review it and suggest changes before real coding starts.

6. What is cost benefit analysis? Explain different areas that come under intangible benefits.

Definition

Cost Benefit Analysis (CBA) is the process of comparing the **costs** of a system/project with its **benefits** to decide whether it is worth implementing.

Formula idea

- If **benefits** > **costs**, project is useful

Types of benefits

- **Tangible benefits**: measurable in money
- **Intangible benefits**: difficult to measure in money

Tangible Benefits

1. Increased Revenue

- **Explanation**: System helps generate more sales
- **Example**: An e-commerce platform like Amazon increases product sales through recommendations

2. Reduced Operating Costs

- **Explanation**: Less money spent on operations
- **Example**: Automating billing reduces need for manual staff

3. Time Savings (Productivity Gain)

- **Explanation**: Tasks are completed faster
- **Example**: A backend system built with Django processes orders instantly instead of manually

4. Reduced Labor Cost

- **Explanation**: Fewer employees required due to automation

- **Example:** Self-checkout systems reduce cashier requirements
-

5. Error Reduction (Cost Saving)

- **Explanation:** Fewer mistakes → less financial loss
 - **Example:** Automated calculations prevent billing errors
-

6. Inventory Cost Reduction

- **Explanation:** Better stock management reduces waste
- **Example:** Real-time inventory system avoids overstocking

Intangible benefit areas

1. Improved customer satisfaction

- Better service quality
- Faster response

2. Better decision-making

- Accurate and timely information

3. Improved employee morale

- Easy systems reduce frustration

4. Better communication

- Easy sharing of information

5. Improved company image

- Professional and reliable reputation

6. Better control and security

- Reduced misuse and better monitoring

7. Higher flexibility

- Easy adaptation to change

8. Competitive advantage

- Better services than competitors

Small points to memorize

- **CBA = compare cost and benefit**
- **Intangible = hard to measure in money**
- Customer satisfaction
- Decision quality
- Employee morale
- Company image
- Security
- Competitive advantage

Real-world application

- A hospital management system may not directly show exact profit, but it improves patient satisfaction, hospital image, and faster decision-making.
-

7. What is outsourcing? What are the advantages and disadvantages of outsourcing?

Definition

Outsourcing is hiring an external company or person to perform some business or IT tasks instead of doing them internally.

Advantages

- Reduces cost
- Access to expert skills
- Saves time
- Focus on core business
- Faster project completion
- Useful for short-term work

Disadvantages

- Less control over work
- Data security risk
- Communication problems
- Quality may vary
- Dependence on third party
- Hidden long-term costs

Small points to memorize

- **Outsourcing** = outside party does the work
- **Advantages** = cost, skill, speed
- **Disadvantages** = control, security, communication

Real-world application

- A startup outsources **website testing** to a freelancer instead of hiring a full-time QA engineer.
-

8. What are the different considerations that must be kept in mind to better design forms and reports?

For forms

- Keep layout simple and neat
- Use clear labels
- Arrange fields in logical order
- Group related items together
- Provide enough space for input
- Use validation to reduce errors
- Highlight mandatory fields
- Keep consistency in font and format
- Make forms user-friendly
- Minimize unnecessary fields

For reports

- Report should have clear title
- Show only relevant information
- Use proper headings and subheadings
- Maintain accuracy and readability
- Use summary where needed
- Keep format consistent
- Highlight important values
- Make it easy to print/store/share
- Provide date, time, and source
- Design according to user needs

Small points to memorize

- Simple
- Clear
- Logical order
- Consistent
- Accurate
- User-friendly
- Relevant info only

Real-world application

- A **hospital patient form** should ask name, age, symptoms, doctor, and contact details in proper order.
 - A **sales report** should show date, product, quantity, total sale, and summary.
-

9. List and define DFD symbols. Draw a DFD up to level-2 for any food ordering system.

DFD symbols

1. External Entity

- Source or destination of data outside the system

Example: Customer, Payment Gateway

2. Process

- Activity that transforms input into output

Example: Place Order, Process Payment

3. Data Flow

- Movement of data from one place to another

Example: Order details, Payment confirmation

4. Data Store

- Place where data is stored

Example: Menu database, Order database

Small points to memorize

- Entity = outside actor

- **Process** = work done
 - **Data flow** = movement
 - **Data store** = saved data
-

Food ordering system DFD

Context Level / Level-0

External entities:

- Customer
- Restaurant
- Payment Gateway

Main process:

- Food Ordering System

Data flow:

- Customer → order details
 - System → order confirmation
 - System ↔ Payment Gateway
 - System → Restaurant → order request/status
-

Level-1 DFD

Processes

1. **Manage Customer Order**
2. **Process Payment**
3. **Send Order to Restaurant**
4. **Update Delivery/Order Status**

Data stores

- D1: Customer Data
- D2: Menu Data
- D3: Order Data
- D4: Payment Data

Flow summary

- Customer sends order → Manage Customer Order
- Manage Customer Order reads Menu Data and stores Order Data
- Payment details → Process Payment → Payment Gateway
- Payment result stored in Payment Data

- Confirmed order → Send Order to Restaurant
 - Restaurant sends order status → Update Delivery Status
 - Final status → Customer
-

Level-2 DFD for “Manage Customer Order”

1.1 Select Items

- Customer chooses food items from menu

1.2 Check Availability

- System checks Menu Data

1.3 Calculate Bill

- System calculates total price, tax, discount

1.4 Create Order

- Order saved in Order Data

1.5 Generate Order Confirmation

- Confirmation sent to customer

Real-world application

- **Swiggy/Zomato type apps** use this flow:
customer selects items → system checks menu → payment done → restaurant receives order
→ status updates shown.
-

10. What is data modeling? Explain conceptual, logical and physical data modeling. Explain normalization up to 3NF.

Data modeling

Data modeling is the process of **designing the structure of data** and defining relationships among data elements in a database.

Types of data modeling

1. Conceptual data model

- High-level model
- Shows major entities and relationships

- No technical details

Example: Student, Course, Teacher

2. Logical data model

- More detailed
- Defines attributes, primary keys, relationships
- Independent of DBMS

Example: Student(StudentID, Name, Class)

3. Physical data model

- Actual implementation in database
- Includes table names, data types, indexes, constraints

Example: StudentID INT PRIMARY KEY, Name VARCHAR(100)

Small points to memorize

- **Conceptual = what data**
 - **Logical = how data is organized**
 - **Physical = how stored in DB**
-

Normalization up to 3NF

1NF (First Normal Form)

- Remove repeating groups
- Each field should have atomic value
- No multiple values in one column

Example problem: one student row has many phone numbers in one field

Fix: separate rows or separate structure

2NF (Second Normal Form)

- Must be in 1NF
- Remove partial dependency
- Non-key attributes should depend on full primary key

Used mainly when primary key is composite

3NF (Third Normal Form)

- Must be in 2NF
- Remove transitive dependency
- Non-key attributes should depend only on primary key, not on other non-key attributes

Example:

StudentID → DeptID

DeptID → DeptName

Then DeptName should be moved to separate department table

Small points to memorize

- 1NF = atomic
- 2NF = no partial dependency
- 3NF = no transitive dependency

Real-world application

In an **e-commerce database**:

- Customer info, order info, and product info should be stored in separate related tables to reduce duplication and errors.
-

11. What are different types of information system? Explain them briefly.

Types of information systems

Types of Information Systems with Real-World Examples

1. Transaction Processing System (TPS)

Handles daily routine transactions (high volume, repetitive work).

Real-world examples:

- Stripe → processes online payments
- PayPal → money transactions
- ATM Machine → cash withdrawal
- Supermarket billing system → barcode scanning & billing

Simple idea:

TPS = Systems that **record daily business transactions**

2. Management Information System (MIS)

Provides reports for middle-level managers (weekly, monthly reports).

Real-world examples:

- Sales report generated from Stripe dashboard
- Inventory report from supermarket system
- Employee attendance report from HR system
- Reports generated from Jira (project progress report)

Simple idea:

MIS = Systems that **generate reports for managers**

3. Decision Support System (DSS)

Helps managers make decisions using data analysis and forecasting.

Real-world examples:

- Sales forecasting system (predict future sales)
- Business analytics dashboards like Power BI
- Google Analytics → website traffic analysis
- System that suggests “Which product should we stock more?”

Simple idea:

DSS = Systems that **help in decision making using data**

4. Executive Information System (EIS)

Provides summarized strategic information for top-level executives (CEO, Director).

Real-world examples:

- CEO Dashboard showing:
 - Total revenue
 - Total profit
 - Growth rate
 - Active users
- Executive dashboards in Tableau
- Company KPI dashboard

Simple idea:

EIS = **Top-level summary for executives**

5. Office Automation System (OAS)

Used for office work, communication, and document management.

Real-world examples:

- Microsoft Word → documents
- Microsoft Excel → spreadsheets

- Gmail → email communication
- Google Docs → document collaboration
- Slack → office communication

Simple idea:

OAS = **Office work tools**

6. Knowledge Management System (KMS)

Stores and shares knowledge inside an organization.

Real-world examples:

- Company documentation portal
- Internal wiki like Confluence
- Notion knowledge base
- Technical documentation website

Simple idea:

KMS = **Company knowledge storage system**

7. Expert System

System that gives expert-level advice using rules and AI.

Real-world examples:

- Medical diagnosis system
- Chatbot that gives legal advice
- Tax calculation software
- AI support bots

Example:

Doctor enters symptoms → system suggests possible disease.

Simple idea:

Expert System = **Computer that thinks like an expert**

Summary Table (Very Important for Exam)

System	Used By	Real Example
TPS	Staff	Stripe, PayPal, ATM
MIS	Middle Manager	Sales Report, Jira Reports
DSS	Manager	Power BI, Google Analytics
EIS	CEO/Top Manager	Tableau Dashboard
OAS	Office Workers	Word, Excel, Gmail, Slack

System	Used By	Real Example
KMS	Employees	Confluence, Notion
Expert System	Specialists	Medical Diagnosis System

2024

2. Define system. Explain components of system including its characteristics.

Definition of System

A **system** is a set of **interrelated components** that work together to achieve a **common goal** by accepting input, processing it, and producing output.

Simple formula

Input → Process → Output

Components of a System

1. Input

The data, material, or resources entered into the system.

Example:

In an ATM system, card details, PIN, and withdrawal amount are inputs.

2. Process

The operation performed on input to convert it into useful output.

Example:

ATM verifies PIN, checks account balance, and processes the withdrawal.

3. Output

The result produced by the system.

Example:

Cash, receipt, and updated balance shown on ATM screen.

4. Feedback

Information returned to control or improve the system.

Example:

If the ATM says “Insufficient balance,” that message is feedback.

5. Control

Rules and procedures that regulate the working of the system.

Example:

ATM allows only limited wrong PIN attempts.

6. Boundary

The limit that defines where the system starts and ends.

Example:

In a school system, students, teachers, exams may be inside the system, but outside vendors may be outside it.

7. Environment

External factors that affect the system.

Example:

Internet connection, power supply, government policy, customer behavior.

8. Interface

The point where two systems or components interact.

Example:

Payment gateway interface between e-commerce site and bank.

Characteristics of a System

- **Organization** – components are arranged in a structure
 - **Interaction** – components communicate with each other
 - **Interdependence** – one part depends on another
 - **Integration** – components work together as a whole
 - **Goal-oriented** – system exists for a specific purpose
 - **Feedback-based** – improves through response and correction
 - **Dynamic** – changes according to needs and environment
-

Real-world application

Hospital Management System

- **Input:** patient details, symptoms, appointment request
 - **Process:** diagnosis, doctor assignment, billing
 - **Output:** prescription, bill, reports
 - **Feedback:** patient response, test results
 - **Control:** access permissions, billing rules
-

3. Define CASE tool. Explain different types of CASE tools that can be used in different phases of SDLC.

Definition of CASE Tool

CASE stands for **Computer-Aided Software Engineering**.

CASE tools are software applications that help automate and support activities in the **Software Development Life Cycle (SDLC)**.

They improve:

- productivity
 - documentation
 - design accuracy
 - testing
 - maintenance
-

Types of CASE Tools in SDLC

1. Upper CASE Tools

Phases Covered:

Planning, Requirement Analysis, System Design

Explanation:

Upper CASE tools are used in the **early stages** of development to analyze requirements and design the system structure.

Functions:

- Requirement specification
- Process modeling (DFD, UML)
- Database and system design

Examples:

- IBM Rational Rose
- Microsoft Visio
- Lucidchart

Example Use:

Designing UML diagrams for a student management system.

2. Lower CASE Tools

Phases Covered:

Implementation, Testing, Maintenance

Explanation:

Lower CASE tools are used in the **later stages** for coding, testing, debugging, and maintaining the system.

Functions:

- Code generation and editing
- Testing and debugging
- System maintenance

Examples:

- Visual Studio Code
- PyCharm
- Selenium

Example Use:

Developing and testing backend APIs.

3. Integrated CASE Tools (I-CASE)

Phases Covered:

All SDLC phases

Explanation:

Integrated CASE tools support the **entire development lifecycle** in a unified environment.

Functions:

- End-to-end development support
- Central repository
- Seamless transition between phases

Examples:

- IBM Rational Suite
- Oracle Designer

Example Use:

Enterprise-level system development with full lifecycle management.

4. Cross-Life Cycle CASE Tools

Phases Covered:

All phases (supporting activities)

Explanation:

These tools support **project management and control activities** throughout the SDLC.

Functions:

- Project planning and tracking
- Version control
- Documentation

Examples:

- GitHub
- Jira
- Confluence

Example Use:

Tracking tasks, managing code versions, and documenting requirements.

Real-world use

A large banking software company uses integrated CASE tools to maintain documentation, design, and testing in one environment.

Other classification by purpose

a. Diagramming tools

Used for DFD, ERD, UML

Examples: draw.io, Visio, StarUML

b. Documentation tools

Used for reports and project documents

Examples: MS Word, Confluence

c. Code generation tools

Generate source code from models

Examples: Visual Studio designer tools

d. Testing tools

Used for software testing

Examples: Selenium, Postman, JMeter, JUnit

e. Configuration management tools

Track versions and changes

Examples: Git, GitHub, GitLab

4. Define information system planning. How is top-down planning different from bottom-up planning approach?

Definition of Information System Planning

Information System Planning (ISP) is the process of identifying how information systems and technology will support an organization's goals, operations, and future growth.

It ensures:

- right system
 - right priorities
 - right investment
 - alignment with business goals
-

Objectives of ISP

- align IT with business strategy

- identify required applications
 - prioritize projects
 - manage resources effectively
 - reduce duplication and risk
-

Top-down Planning Approach

Meaning

Planning starts from **top management** and overall business goals, then moves downward to departments and systems.

Features

- strategy-driven
- long-term focus
- organization-wide view
- strong management control

Advantages:

- Clear direction and vision
- Better alignment with organizational goals
- Easier control and coordination

Disadvantages:

- May ignore practical issues at lower levels
- Less employee involvement
- Can be rigid

Real-world example

A university first decides its digital transformation strategy, then develops modules for admission, exams, library, and hostel.

Bottom-up Planning Approach

Meaning

Planning starts from **operational levels or departments**, then moves upward to broader organizational systems.

Features

- department-focused
- practical and immediate

- problem-solving oriented

Advantages:

- More realistic and practical plans
- Encourages participation and innovation
- Better acceptance by employees

Disadvantages:

- May lack overall direction
- Time-consuming
- Difficult to coordinate

Real-world example

The accounts department first develops payroll software, then other departments create separate systems later.

Difference between Top-down and Bottom-up

Basis	Top-down	Bottom-up
Starting point	Top management	Lower departments/users
Focus	Strategic goals	Operational needs
Scope	Organization-wide	Department-wise
Integration	High	May be low
Speed	Slower	Faster
Best for	Large planned systems	Quick departmental solutions

Real-world application

In an **e-commerce company**:

- **Top-down:** company plans one integrated platform for inventory, sales, payment, and analytics
 - **Bottom-up:** warehouse team first creates only stock tracking software
-

5. Define JAD. List different contemporary requirements determining techniques and explain them.

Definition of JAD

JAD (Joint Application Development) is a structured method of gathering system requirements through meetings/workshops involving:

- users
- managers
- analysts
- developers

The goal is to define requirements quickly and accurately.

Features of JAD

- group discussion
 - active user participation
 - fast requirement collection
 - reduced misunderstanding
 - better agreement among stakeholders
-

Real-world use of JAD

Before creating a hospital software system, doctors, receptionists, administrators, and developers meet together to decide features like appointments, billing, and patient records.

Contemporary Requirement Determination Techniques

1. Interviews

Analyst asks users and managers direct questions.

Advantages

- detailed information
- can clarify doubts
- useful for complex systems

Example

Interviewing a bank manager to understand loan approval workflow.

2. Questionnaires / Surveys

A written set of questions distributed to many users.

Advantages

- low cost
- reaches many people
- easy for large organizations

Example

Sending a Google Form to students for feedback on college ERP needs.

Tools

- Google Forms
 - Microsoft Forms
 - SurveyMonkey
-

3. Observation

Analyst watches users performing tasks.

Advantages

- shows actual work
- helpful when users cannot explain clearly

Example

Watching a cashier use POS software in a supermarket.

4. Document Analysis

Studying existing forms, reports, files, manuals, and procedures.

Advantages

- identifies current workflow
- useful in old/manual systems

Example

Studying invoices, receipts, and ledger books before automating accounting.

5. Prototyping

Creating a sample model of the system for feedback.

Advantages

- clears confusion
- users can see early version
- reduces requirement errors

Example

Building a rough app screen for food delivery before full development.

Tools

- Figma
 - Adobe XD
 - Balsamiq
-

6. JAD Sessions

Group workshop for discussion and requirement finalization.

Advantages

- quick decisions
 - participation from all sides
 - fewer conflicts later
-

7. Brainstorming

Group members suggest ideas freely.

Advantages

- encourages creativity
- useful for innovative projects

Example

Brainstorming features for a stock market analytics platform.

8. Use Cases / User Stories

Describes how users interact with the system.

Example

“As a customer, I want to book a hotel room online so that I can confirm my stay.”

Tools

- Jira
 - Trello
 - Azure DevOps
-

9. Agile Requirement Workshops

Short, repeated discussions during development.

Advantages

- suitable for changing requirements
 - continuous user feedback
-

6. Define data modeling. Explain logical data model in detail.

Levels of data modeling

- Conceptual
 - Logical
 - Physical
-

1. Conceptual Data Model (High-Level)

Definition:

The conceptual model represents the **overall structure of data** without technical details. It focuses on *what* data exists and *how entities relate*.

Key Points:

- Very abstract (business view)
- No database details
- Defines **entities** and **relationships** only
- Used for communication with stakeholders

Example:

In an online store:

- Entities: Customer, Product, Order
- Relationship: Customer places Order

Tools:

- Lucidchart
 - Microsoft Visio
-

2. Logical Data Model (Detailed Structure)

Definition:

The logical model describes the **structure of data in detail**, but still independent of any specific database system.

Key Points:

- Includes **attributes (fields)**
- Defines **primary keys and relationships**
- Normalization is applied
- Technology-independent

Example:

Customer entity becomes:

- Customer(ID, Name, Email)

Order entity:

- Order(ID, Date, CustomerID)

Tools:

- MySQL Workbench
 - ERwin Data Modeler
-

3. Physical Data Model (Implementation Level)

Definition:

The physical model defines **how data is actually stored in the database system**.

Key Points:

- Includes **tables, columns, data types**
- Database-specific (MySQL, PostgreSQL, etc.)
- Includes indexes, constraints, storage details

Example:

```
CREATE TABLE customer (  
  id INT PRIMARY KEY,
```

```
name VARCHAR(100),
email VARCHAR(100)
);
```

Tools:

- PostgreSQL
- MySQL

◆ Key Differences (Exam Table)

Level	Focus	Detail Level	Technical?	Users
Conceptual	What data & relations	Low	No	Managers, analysts
Logical	Structure of data	Medium	Partial	Designers
Physical	Storage in database	High	Yes	Developers, DBAs

Logical Data Model

A **logical data model** describes the structure of data in detail without depending on a specific DBMS.

It focuses on:

- entities
- attributes
- primary keys
- relationships
- business rules

Elements of Logical Data Model

1. Entity

A real-world object or concept.

Examples: Student, Customer, Order, Product

2. Attribute

A property of an entity.

Example:

Customer → CustomerID, Name, Phone, Address

3. Primary Key

A unique identifier for each record.

Example:

CustomerID

4. Foreign Key

An attribute that connects two entities.

Example:

Order table contains CustomerID as foreign key

5. Relationship

Shows how entities are connected.

Types

- one-to-one
- one-to-many
- many-to-many

Example:

One customer can place many orders

6. Constraints / Business Rules

Rules that data must follow.

Example:

A room booking cannot have check-out date earlier than check-in date.

Characteristics of Logical Data Model

- DBMS independent
 - detailed but not technical like physical model
 - normalized structure
 - defines relationships clearly
 - useful for developers and analysts
-

Example: Online Shopping System

Entities

- Customer
- Product
- Order
- Payment

Relationships

- Customer places Order
- Order contains Product
- Order has Payment

Sample logical design

- Customer(CustomerID, Name, Email, Phone)
 - Product(ProductID, ProductName, Price, Stock)
 - Order(OrderID, OrderDate, CustomerID)
 - Payment(PaymentID, OrderID, Amount, PaymentMethod)
-

Tools used

- ERwin
 - MySQL Workbench
 - Lucidchart
 - dbdiagram
 - draw.io
-

Real-world application

In an **e-commerce database**, logical modeling ensures customer, order, and payment data are connected properly and duplication is reduced.

7. What are the various types of menu design which can be adopted to meet system complexity and usability?

Meaning of Menu Design

Menu design refers to the arrangement of options in a system interface so users can perform tasks easily.

Types of Menu Design

1. Single Menu

Only one menu is shown with all options.

Best for

Simple systems

Example

Calculator app main menu

2. Sequential Menu

User moves through options step by step.

Best for

Processes requiring order

Example

ATM cash withdrawal steps

3. Hierarchical Menu

Main menu leads to submenus.

Best for

Large systems with many functions

Example

Settings in Android phone

4. Top/Navigation Bar Menu

Options are displayed horizontally at top.

Best for

Websites and dashboards

Example

Home, About, Services, Contact

5. Pop-up / Context Menu

Menu appears when needed.

Best for

Quick actions

Example

Right-click menu in Windows

6. Pull-down / Drop-down Menu

List appears when clicked.

Best for

Saving screen space

Example

Country list in a registration form

7. Mega Menu

Large panel with grouped options.

Best for

Complex websites

Example

Amazon category menu

8. Iconic Menu

Uses icons instead of text or along with text.

Best for

Mobile apps and visual systems

Example

Camera, gallery, settings icons

9. Hamburger Menu

Collapsed menu icon with hidden options.

Best for

Mobile responsive websites

Example

Menu in many mobile web apps

10. Ribbon Menu

Commands grouped in tabs.

Best for

Feature-rich desktop software

Example

MS Word ribbon

Usability considerations

- menu should be simple
- use meaningful labels
- avoid too many options in one place
- group related items

- maintain consistency
 - provide shortcuts where useful
-

Real-world application

A **banking mobile app** may use:

- bottom navigation menu for main features
 - hamburger menu for extra options
 - drop-down menu for account selection
-

8. Why do software projects often fail? Explain different types of software testing.

Why software projects often fail

1. Unclear requirements

If requirements are incomplete or frequently changing, the project becomes unstable.

2. Poor planning

Wrong budget, wrong timeline, weak resource allocation.

3. Lack of user involvement

Users are not consulted, so the final system does not meet their needs.

4. Weak communication

Misunderstanding among client, analyst, developer, and tester.

5. Technical complexity

New technologies or poor architecture increase risk.

6. Scope creep

More and more features are added without proper control.

7. Poor risk management

No backup plan for delays, failures, or dependency issues.

8. Lack of testing

Bugs remain in the software.

9. Inexperienced team

Team may lack domain knowledge or technical skill.

10. Poor project management

No tracking, no prioritization, no timely decisions.

Different types of software testing

1. Unit Testing

Tests individual functions/modules.

Example: testing login function only

Tools: pytest, JUnit, unittest

2. Integration Testing

Tests interaction between modules.

Example: testing whether login module works with database

3. System Testing

Tests the complete system as a whole.

Example: testing full e-commerce system from sign-up to payment

4. Acceptance Testing

Checks whether software meets user requirements.

Types

- Alpha testing
 - Beta testing
 - UAT (User Acceptance Testing)
-

5. Regression Testing

Retests software after changes to ensure old features still work.

Example: after adding coupon feature, ensure payment still works

Tools: Selenium, Cypress

6. Functional Testing

Checks whether software functions according to requirements.

7. Non-functional Testing

Checks quality attributes like:

- performance
 - security
 - usability
 - reliability
-

8. Performance Testing

Checks speed and stability under load.

Tools: JMeter, Locust

9. Security Testing

Checks vulnerabilities and threats.

Tools: OWASP ZAP, Burp Suite

10. Usability Testing

Checks ease of use for real users.

11. Smoke Testing

Basic test to check whether build is stable enough for further testing.

12. Black-box Testing

Tests without knowing internal code.

13. White-box Testing

Tests internal code logic and paths.

Real-world application

For a **FastAPI stock app**:

- unit test API endpoints
 - integration test database and API
 - performance test live market data calls
 - acceptance test dashboard behavior
-

9. Explain phases of SDLC. Also discuss the role of Project Manager in software project development.

Phases of SDLC

1. Planning

Identify problem, goals, budget, scope, timeline, feasibility.

Output: project plan

2. Requirement Analysis

Gather and analyze user needs.

Output: SRS document

Tools: interviews, JAD, Jira, Confluence

3. System Design

Create architecture, database design, interface design, module design.

Output: design documents, diagrams

Tools: Figma, draw.io, StarUML, MySQL Workbench

4. Development / Coding

Actual coding of the software.

Tools: VS Code, PyCharm, Git, GitHub

5. Testing

Find and fix defects.

Tools: pytest, Selenium, Postman, JMeter

6. Implementation / Deployment

Install software in real environment and train users.

Tools: Docker, AWS, Nginx, CI/CD pipelines

7. Maintenance

Correct, adapt, improve, and prevent future problems.

Role of Project Manager

Main responsibilities

- project planning
- team coordination
- resource allocation
- risk management
- communication with client
- monitoring schedule and cost
- quality assurance
- change management

During each phase

- **Planning:** define scope, timeline, cost
- **Analysis:** ensure proper requirement gathering
- **Design:** review architecture decisions
- **Development:** track team progress
- **Testing:** ensure quality targets
- **Deployment:** coordinate release
- **Maintenance:** manage support and updates

Real-world application

In a **hotel booking app**, the project manager ensures developers, designers, testers, and client all remain aligned and the project is delivered on time.

10. What is process modeling? Why do we need DFD? Draw up to top level DFD for Online hotel room booking system.

Process Modeling

Process modeling is the representation of business processes and data movement in a system to understand how work is done.

It shows:

- what activities happen
 - how data flows
 - who interacts with the system
-

Why do we need DFD?

A **Data Flow Diagram (DFD)** is used to show flow of data in a system.

Importance

- easy to understand
 - helps in requirement analysis
 - identifies inputs, outputs, stores, and processes
 - useful for communication with users and developers
 - reduces design confusion
-

DFD Symbols

- **External Entity** – source/destination outside system
 - **Process** – activity
 - **Data Flow** – movement of data
 - **Data Store** – stored data
-

Online Hotel Room Booking System DFD

Context / Top Level DFD (Level-0)

External Entities

- Customer
- Hotel Admin
- Payment Gateway

Main Process

- Online Hotel Room Booking System

Data flows

- Customer → search request, booking details, payment details
 - System → room availability, booking confirmation, invoice
 - System ↔ Payment Gateway → payment request / payment status
 - Hotel Admin → room details, pricing, availability updates
 - System → booking reports/status to Hotel Admin
-

Level-1 DFD

Processes

1. Search Rooms
2. Manage Booking
3. Process Payment
4. Update Room Availability
5. Generate Confirmation

Data Stores

- D1 Customer Data
- D2 Room Data
- D3 Booking Data
- D4 Payment Data

Flow

- Customer sends search criteria → Search Rooms → reads Room Data → shows available rooms
- Customer selects room → Manage Booking → stores Booking Data
- Payment details → Process Payment → Payment Gateway
- Payment result stored in Payment Data
- Booking confirmation → Generate Confirmation → Customer
- Hotel Admin updates room and price → Update Room Availability → Room Data

Real-world application

This is used in **OYO, Booking.com, Agoda**-type systems where customer searches rooms, books, pays, and receives confirmation.

11. List different activities associated with implementation phase. Explain methods of training and supporting users.

Activities in Implementation Phase

1. Installation of hardware and software

Required machines, OS, DBMS, application software are installed.

Example: installing Linux server, PostgreSQL, and hospital management software

2. Data Conversion / Data Migration

Old data is moved into the new system.

Example: transferring student records from Excel to ERP

3. System Testing in live environment

Check whether system works properly in actual conditions.

4. User Training

Users are trained to operate the system.

5. Documentation

Prepare manuals, guides, help documents, SOPs.

6. Changeover / Conversion Method

Switch from old system to new system.

Types of changeover

- **Direct changeover** – old system stops, new starts immediately
 - **Parallel changeover** – both old and new run together for some time
 - **Pilot changeover** – new system used in one department first
 - **Phased changeover** – introduced module by module
-

7. User Acceptance

Final approval from users.

8. Support and Maintenance Setup

Provide help desk, bug fixing, monitoring, and updates.

Methods of Training Users

1. Classroom Training

Trainer teaches a group of users.

Best for: large organizations

2. Hands-on Training

Users practice directly in the system.

Best for: practical learning

3. One-to-One Training

Trainer teaches one user personally.

Best for: managers or special roles

4. Online / Video Training

Training through recorded videos or live sessions.

Tools: Zoom, Google Meet, LMS

5. Training Manuals / User Guides

Written instructions with screenshots.

6. Train-the-Trainer Method

A few users are trained first; they train others later.

Best for: large distributed organizations

Methods of Supporting Users

1. Help Desk

Users report issues and get assistance.

Tools

- Freshdesk
 - Zendesk
 - Jira Service Management
-

2. User Manual / FAQ

Provides solutions to common problems.

3. On-site Support

Technical staff help users directly at location.

4. Remote Support

Support provided using phone, chat, email, or remote desktop.

Tools: AnyDesk, TeamViewer

5. Maintenance and Updates

Regular bug fixes and improvements.

6. Knowledge Base / Self-service Portal

Users search solutions themselves.

Real-world application

When a **college ERP system** is implemented:

- staff are trained with demos and manuals
 - student data is migrated
 - pilot rollout begins in one department
 - help desk supports teachers and admin staff
-

Very short memory points

2. System

Input, process, output, feedback, control, boundary, environment, interface

3. CASE tools

Upper CASE, Lower CASE, Integrated CASE

4. ISP

Planning IS for business goals

Top-down = strategic

Bottom-up = departmental

5. JAD

Group requirement workshop

Techniques: interview, questionnaire, observation, prototype, document analysis, user stories

6. Data modeling

Logical model = entities, attributes, keys, relationships, rules

7. Menu design

Single, sequential, hierarchical, drop-down, pop-up, iconic, mega, hamburger

8. Failure + Testing

Failure: poor requirements, planning, communication

Testing: unit, integration, system, acceptance, regression, performance, security

9. SDLC

Planning, analysis, design, development, testing, implementation, maintenance

10. Process modeling / DFD

Shows process and data flow in system

11. Implementation

Install, migrate data, test, train, document, changeover, support

1) Method Choice: Waterfall vs Agile for Air Traffic Control

For a **large government air traffic control (ATC) system**, I would choose a **Waterfall (plan-driven / V-model) approach**, not pure Agile.

Why Waterfall fits better

- **Safety-critical domain:** ATC failures can risk lives → requires **formal verification, validation, and traceability**.
- **Stable, regulated requirements:** Aviation standards demand **fixed, auditable requirements** and strict compliance.
- **Heavy documentation:** Certification bodies require complete artifacts (SRS, design specs, test evidence).
- **Predictability & governance:** Government projects need **clear milestones, budgets, and approvals**.

How to use it practically

Use **Waterfall (or V-Model)** as the backbone, but incorporate **Agile practices within phases** (e.g., iterative prototyping for UI, frequent demos) without compromising compliance.

One-line exam answer

For safety-critical, highly regulated systems like air traffic control, a **Waterfall/V-Model** approach is preferred due to its emphasis on **documentation, traceability, and rigorous testing**, though limited Agile practices can be embedded for flexibility.

2) Role & Responsibilities of a Software Project Manager

A **Software Project Manager (SPM)** is accountable for delivering the system **on time, within budget, and to required quality**.

Core Responsibilities

1. **Project Planning**
 - Define scope, WBS, schedule, and milestones
 - Create the baseline plan (scope–time–cost)
2. **Resource Management**
 - Allocate developers, testers, tools, infrastructure
 - Balance workload and capacity
3. **Risk Management**
 - Identify risks (technical, schedule, regulatory)
 - Mitigation and contingency planning
4. **Quality Management**
 - Ensure adherence to standards (reviews, testing, audits)
 - Define acceptance criteria
5. **Communication & Stakeholder Management**
 - Coordinate with clients, regulators, and team
 - Status reporting and expectation management
6. **Cost & Budget Control**
 - Track expenses vs budget
 - Control scope creep and cost overruns
7. **Monitoring & Control**
 - Track progress (KPIs, milestones)
 - Take corrective actions
8. **Compliance & Documentation**
 - Ensure required documents and certifications are complete

One-line exam answer

The SPM plans, organizes, leads, and controls project activities, ensuring delivery within **scope, time, cost, and quality constraints**, while managing risks, resources, and stakeholders.

3) Payback Period (with 6% interest)

Given:

- Initial investment = ₹2,00,000
- Annual profit (cash inflow) = ₹50,000
- Interest rate = 6%

Step 1: Simple Payback (ignoring interest)

$\text{years Payback} = 50,000 / 200,000 = 4 \text{ years}$

Step 2: Discounted Payback (considering 6%)

Present value (PV) of yearly inflows:

- Year 1: $50,000 / 1.06 = 47,170$
- Year 2: $50,000 / 1.06^2 \approx 44,500$
- Year 3: $50,000 / 1.06^3 \approx 41,981$
- Year 4: $50,000 / 1.06^4 \approx 39,604$
- Year 5: $50,000 / 1.06^5 \approx 37,360$

Cumulative PV:

- End of Year 4 $\approx 1,73,255$
- Remaining to recover $\approx 26,745$
- Fraction of Year 5 needed $\approx 26,745 / 37,360 \approx 0.72$

Final Answer:

- **Simple Payback Period: 4 years**
- **Discounted Payback Period (6%): ≈ 4.7 years (about 4 years 9 months)**

3. Roles & Responsibilities of Software Project Manager

◆ Roles:

- Planner
- Leader
- Coordinator
- Decision-maker

◆ Responsibilities:

1. **Project Planning**
 - Define scope, schedule, budget
2. **Resource Management**
 - Assign team and tools
3. **Risk Management**
 - Identify and handle risks
4. **Quality Assurance**
 - Ensure system meets standards
5. **Communication**
 - Coordinate with stakeholders
6. **Monitoring & Control**
 - Track progress and performance

◆ Conclusion:

A project manager ensures the project is completed **on time, within budget, and with required quality**.

Types of System Implementation

System implementation is the process of **deploying a new system** and making it operational in an organization. The strategy chosen affects **risk, cost, and continuity of operations**.

◆ ◆ 1. Direct Implementation (Big Bang)

Definition:

The old system is completely replaced by the new system at once.

Real-world example:

A company like **a small e-commerce startup (e.g., a new online clothing brand)** replaces its old manual Excel-based order system with a **new Shopify-based order management system overnight**. From the next day, all orders are processed only through Shopify.

Key idea:

One-time full switch → no old system after cutover.

◆ 2. Parallel Implementation

Definition:

Old and new systems run simultaneously for a period.

Real-world example:

A **bank like Nepal Bank Limited or any commercial bank** introduces a new digital banking system while still maintaining the old core banking system. For some months, transactions are recorded in both systems to verify accuracy.

Key idea:

Double system operation for safety and comparison.

◆ 3. Phased Implementation

Definition:

The system is introduced step-by-step in modules.

Real-world example:

A large company like **Amazon or Flipkart** upgrades its system gradually:

- First rollout: user login system update
- Second rollout: payment gateway integration
- Third rollout: delivery tracking system

Each module is deployed separately over time.

Key idea:

Gradual upgrade of different parts of the system.

◆ 4. Pilot Implementation

Definition:

New system is tested in one department or location before full rollout.

Real-world example:

A bank like **Himalayan Bank or SBI (State Bank of India)** first implements a new ATM software system in **one branch (e.g., Kathmandu branch or Mumbai branch)**. After successful testing and feedback, it is expanded to all branches.

Key idea:

Test in small area first, then scale globally.

◆ Comparison Table (Exam Ready)

Method	Risk Level	Cost	Speed	Best Use Case
Direct	High	Low	Fast	Small/simple systems
Parallel	Low	High	Slow	Critical systems (banking)
Phased	Medium	Medium	Moderate	Large modular systems
Pilot	Low	Medium	Slow	Large organizations

System Testing – Definition

System testing is the phase where the **complete, integrated software system** is tested to verify that it meets all specified functional and non-functional requirements. It is typically performed **after integration testing** and before **user acceptance testing (UAT)**.

Why it matters

- Validates **end-to-end workflows** (not just modules)
 - Checks **real user scenarios**
 - Ensures **quality, reliability, and compliance**
-

Types of System Testing (with examples & tools)

1. Functional Testing

What it checks: All features work as per requirements.

Example:

In an e-commerce app (like Amazon):

- Add to cart → Checkout → Payment → Order confirmation

Tools:

Selenium, Postman

2. Performance Testing

What it checks: Speed, scalability, and stability under load.

Example:

- 10,000 users access the site simultaneously → system should not crash

Tools:

Apache JMeter

3. Security Testing

What it checks: Protection against unauthorized access and vulnerabilities.

Example:

- Prevent SQL injection in login form
- Ensure user passwords are encrypted

Tools:

OWASP ZAP

4. Usability Testing

What it checks: Ease of use and user experience.

Example:

- Users can easily navigate menus and complete checkout

Real-world: Apps like Instagram focus heavily on usability

5. Compatibility Testing

What it checks: Works across devices, browsers, and OS.

Example:

- Website works on Chrome, Safari, Android, iOS

◆ 1NF (First Normal Form)

◆ 2NF (Second Normal Form)

👉 Applies when there is a **composite key**

◆ 3NF (Third Normal Form)

Quick Understanding (Very Important)

Normal Form	What is NOT Allowed	Fix
1NF	Multiple values in one column	Make values atomic
2NF	Partial dependency	Split into separate tables
3NF	Transitive dependency	Remove indirect dependency

What is Normalization?

Normalization is the process of organizing data in a database to **reduce redundancy (duplicate data)** and **improve data integrity** by dividing data into related tables and defining relationships.

◆ Objectives of Normalization

- Eliminate duplicate data
 - Ensure data consistency
 - Prevent anomalies (insert, update, delete)
 - Improve efficiency of database design
-

◆ Types (Normal Forms)

1. First Normal Form (1NF)

Rule:

- No repeating groups or multivalued attributes
- Each field contains **atomic (single) values**

Example (Before 1NF):

✗ NOT in 1NF (has repeating / multivalued data)

Student

ID	Name	Subjects

1	James	Math, Science
2	Ram	English, Nepali

● Problem:

- Subjects has multiple values in one column (not atomic)
-

✓ In 1NF

Student

ID	Name	Subject

1	James	Math
1	James	Science
2	Ram	English
2	Ram	Nepali

✓ Each field has **single (atomic) value**

2. Second Normal Form (2NF)

Rule:

- It is in 1NF
 - All non-key attributes depend on the **entire primary key**
-

Example (Before 2NF):

✗ NOT in 2NF (partial dependency)

OrderDetails

OrderID	ProductID	ProductName	Quantity
101	P1	Laptop	2
101	P2	Mouse	1

● Problem:

- Primary Key = (OrderID, ProductID)
 - ProductName depends only on ProductID (partial dependency)
-

✓ In 2NF

OrderDetails

OrderID	ProductID	Quantity
101	P1	2
101	P2	1

Product

ProductID	ProductName
P1	Laptop
P2	Mouse

✓ Removed **partial dependency**

3. Third Normal Form (3NF)

Rule:

A table is in 3NF if:

- It is in 2NF
- There is **no transitive dependency**
-

✗ NOT in 3NF (transitive dependency)

Employee

EmpID	DeptID	DeptName
1	D1	HR
2	D2	IT

● Problem:

- DeptName depends on DeptID (not directly on EmpID)

✓ In 3NF

Employee

EmpID	DeptID
1	D1
2	D2

Department

DeptID	DeptName
D1	HR
D2	IT

✓ Removed transitive dependency

4. Boyce-Codd Normal Form (BCNF)

Rule:

- Stronger version of 3NF
- Every determinant must be a candidate key

Example:

Used in complex cases where 3NF is not sufficient.

◆ Summary Table (Exam Ready)

Normal Form	Condition	Purpose
1NF	Atomic values, no repetition	Remove repeating groups
2NF	No partial dependency	Remove partial dependency
3NF	No transitive dependency	Remove indirect dependency

Normal Form	Condition	Purpose
BCNF	Stronger than 3NF	Handle complex anomalies

2. Meaning of Deliverable

A **deliverable** is a **tangible or measurable output** produced during a project that is submitted to stakeholders or clients.

✓ Examples:

- SRS document
- Design document
- Software application
- Test reports
- User manual

👉 In simple terms: deliverable = **what the project produces and hands over.**

◆ 3. Major Deliverables of Planning Phase

In the planning phase of SDLC, the main deliverables are:

✓ 1. Project Plan

- Defines schedule, tasks, milestones, and timeline.

✓ 2. Feasibility Study Report

- Evaluates whether the project is technically, economically, and operationally possible.

✓ 3. Resource Plan

- Identifies required manpower, hardware, and software.

✓ 4. Cost Estimation/Budget Plan

- Estimates total project cost.

✓ 5. Risk Management Plan

- Identifies risks and mitigation strategies.

✓ 6. Scope Statement

- Defines what is included and excluded in the project.

👉 In simple terms: planning phase deliverables = **blueprint of the project before development starts.**

A DFD (Data Flow Diagram) is a graphical representation that shows **how data flows through a system** and how it is processed.

Exam Definition

A Data Flow Diagram (DFD) is a diagram that illustrates the flow of data between processes, data stores, and external entities in a system.

Purpose of DFD

- To understand how a system works
 - To visualize data movement
 - To simplify complex processes
 - To help in system analysis and design
-

Main Components of DFD

1. External Entity

- Source or destination of data
 Example: Customer, Bank
-

2. Process

- Activity that transforms data
 Example: “Process Order”
-

3. Data Flow

- Movement of data (arrow)
 Example: Order details

Types of Attributes

1. Simple Attribute (Atomic Attribute)

- Cannot be divided further
 Example: Age, Salary

 Single value, no sub-parts

2. Composite Attribute

- Can be divided into smaller sub-parts

👉 Example:

Name → First Name + Last Name

Address → City + State + Zip

3. Single-Valued Attribute

- Holds only one value for each entity

👉 Example: Roll Number (one student = one roll number)

4. Multi-Valued Attribute

- Can have multiple values

👉 Example: Phone Number (one person = many numbers)

5. Derived Attribute

- Value is calculated from other attributes

👉 Example:

Age derived from Date of Birth

1. What is System Requirement Analysis? Explain types of determining system requirements.

 **Answer**

System Requirement Analysis

System Requirement Analysis is the process of **collecting, analyzing, and defining the needs and requirements of a system** before development.

👉 It ensures the system meets user expectations and avoids errors later.

Types (Methods) of Determining Requirements

1. Interviews

- Direct discussion with users and stakeholders
 - Provides detailed information
-

2. Questionnaires

- Written set of questions
 - Useful for large number of users
-

3. Observation

- Analyst observes actual work environment
 - Helps understand real processes
-

4. Document Analysis

- Study existing forms, reports, files
 - Understand current system
-

5. JAD (Joint Application Development)

- Group discussion between users and developers
 - Faster decision-making
-

6. Prototyping

- Build a sample system
 - Users give feedback
-

2. What qualities should a software tester have?

Answer

A good software tester should have:

-  **Attention to detail** – find small errors
 -  **Analytical thinking** – understand system logic
 -  **Testing knowledge** – methods and tools
 -  **Communication skills** – report bugs clearly
 -  **Patience** – testing takes time
 -  **Problem-solving ability** – identify root cause
 -  **Domain knowledge** – understand system use
-

3. What are deliverables from coding, testing, and installation?

Answer

Coding Deliverables

- Source code
 - Program modules
 - Code documentation
-

Testing Deliverables

- Test cases
 - Test reports
 - Bug/defect reports
-

Installation Deliverables

- Installed system
 - User manuals
 - Configuration files
 - Installation guide
-

4. Explain different types of software testing in detail.

Answer

1. Unit Testing

- Tests individual components/modules
-

2. Integration Testing

- Tests interaction between modules
-

3. System Testing

- Tests complete system

◆ 4. Acceptance Testing

- Done by users to verify system

◆ 5. Regression Testing

- Ensures new changes don't break system

◆ 6. Performance Testing

- Checks speed and response time

◆ 7. Load Testing

- Tests system under normal load

◆ 8. Stress Testing

- Tests system under extreme conditions

◆ 9. Security Testing

- Checks protection from threats

◆ 10. Usability Testing

- Tests user-friendliness
-

5. Why do we draw DFD?

Answer

We draw **DFD (Data Flow Diagram)** to:

- Understand system clearly
- Show flow of data
- Simplify complex processes
- Help in system design
- Improve communication between users and developers

6. Process of Drawing DFD (with Example)

Answer

Steps:

1. Identify External Entities

 Example: Customer

2. Identify Processes

 Example: Place Order

3. Identify Data Stores

 Example: Order Database

4. Identify Data Flows

 Example: Order Details

5. Draw Level 0 (Context Diagram)

- Show overall system

6. Draw Level 1

- Break system into main processes

7. Draw Level 2

- Detailed breakdown

Example (Food Ordering System)

- Customer → Place Order → Database
 - System → Send Order → Restaurant
-

7. Describe Integrated CASE Tools with application.

Answer

What is CASE Tool?

CASE (Computer-Aided Software Engineering) tools are software that help in **designing, developing, and maintaining** systems.

Integrated CASE Tools

Integrated CASE tools support **multiple phases of SDLC in one system**.

 Example tools:

- Modeling + coding + testing in one platform
-

Features

- Diagram creation (DFD, ERD)
 - Code generation
 - Documentation support
 - Testing support
-

Applications

- System analysis and design
 - Database design
 - Software development
 - Project management
-

Final Tip

For exams:

- Write **headings clearly**
- Use **points + examples**
- Draw **small diagrams (DFD)** if asked